

02 API - Gegevens toevoegen, wijzigen en verwijderen

📅 09-09-2026 ⌚ Ongeveer 18 minuten 🗨️ Ongeveer 2701 woorden

In het vorige hoofdstuk leerde je hoe je gegevens uit een database kunt ophalen met behulp van een API. In dit hoofdstuk leer je hoe je ook gegevens kunt toevoegen aan, wijzigen en verwijderen uit de database. Maar eerst wordt er dieper ingegaan op de methodes om gegevens op te halen.

LINQ

In tegenstelling tot Dapper hoeft je in Entity Framework geen SQL te gebruiken maar gebruik je LINQ, wat staat voor Language Integrated Query. LINQ is een feature van .NET waarmee je, op een leesbare manier, query's kunt uitvoeren op array's, collecties en ook DbSet's. LINQ ondersteunt Intellisense en LINQ-Queries worden gecontroleerd op typefouten. Op die manier kun je runtime-fouten voorkomen.

Enkele veelgebruikte LINQ-operaties zijn:

- **Select:** Kies specifieke velden of transformeer de output.
- **Where:** Filter resultaten op basis van een voorwaarde.
- **OrderBy / OrderByDescending:** Sorteer de resultaten.
- **GroupBy:** Groepeer resultaten op basis van een sleutel.
- **First / FirstOrDefault:** Haal het eerste element op dat voldoet aan een bepaalde voorwaarde. First geeft echter een *InvalidOperationException* als er geen element gevonden wordt. FirstOrDefault geeft in dat geval de default-waarde terug.
- **Any:** Controleer of er ten minste één element voldoet aan een bepaalde voorwaarde. Je krijgt hier een true or false terug.
- **All:** Controleer of alle elementen voldoen aan een bepaalde voorwaarde.
- **Count:** Tel het aantal elementen dat voldoet aan een voorwaarde.
- **Find:** Haalt een entiteit op op basis van de primaire sleutel in Entity Framework.

Voor enkele van deze methoden bestaat ook een async-versie. Gebruik deze versies bij grotere databaseoperaties om te voorkomen dat je applicatie blokkeert. Op die manier blijft je gebruikersinterface responsief en kan op de achtergrond de data verwerkt worden. Als je asynchrone methodes gebruikt, moet je van de ActionResult-functie een **Task** maken:

1

```
public async Task<ActionResult<Product>> GetProduct(int id)
```

CS

Syntax: Context.Entiteit(of repository).Methode1.Methode2

Het principe van methode achter elkaar te zetten, wordt *Method Chaining* genoemd. Als je na een methode of object een punt typt, krijg je een lijst van de nieuwe beschikbare methodes.

In een LINQ-methode wordt, in deze cursus, steeds een lambda-expressie gebruikt om je query uit te drukken. Een lambda-expressie is een compacte manier om een functie te schrijven.

Hieronder vind je een voorbeeld van een GET-Request om een deel van de productnaam te zoeken. Je krijgt als resultaat een gesorteerde lijst van alle producten die voldoen aan een zoekwaarde. `x=>x.Naam.Contains(zoekwaarde)` is een voorbeeld van een lambda-expressie.

LINQ

```
1 [HttpGet("Search")]
2 public async Task<ActionResult<List<Product>>> Search(string zoekwaarde)
3 {
4     var producten = await
5     _context.Producten.Where(x=>x.Naam.Contains(zoekwaarde)).OrderBy(x=>x.Naam
6     if (producten == null)
7     {
8         return NotFound($"Er zijn geen producten in de database waar {zoekwaarde}
9     in de naam");
10    }
11    return Ok(producten);
12 }
```

CS

SQL

```
1 [HttpGet("Search")]
2 public async Task<ActionResult<List<Product>>> Search(string
3 zoekwaarde)
4 {
5     var producten = await _context.Producten
6     .FromSqlRaw("SELECT * FROM Producten WHERE Naam LIKE {0} ORDER
```

CS

```
7     BY Naam", $"{zoekwaarde}%")
8         .ToListAsync();
9     if (producten == null)
10    {
11        return NotFound($"Er zijn geen producten in de database waar
12    {zoekwaarde} voorkomt in de naam");
13    }
14    return Ok(producten);
15 }
```

extra voorbeelden van LINQ

LINQ

```
1 // 1. Select: Kies specifieke velden of transformeer output
2 List<string> namen = await _context.Producten
3     .Select(p => p.Naam)
4     .ToListAsync();
5
6 // 2. Where: Filter resultaten op basis van een voorwaarde
7 List<Product> dureProducten = await _context.Producten
8     .Where(p => p.Prijs > 100)
9     .ToListAsync();
10
11 // 3. OrderBy / OrderByDescending: Sorteer de resultaten
12 List<Product> gesorteerd = await _context.Producten
13     .OrderByDescending(p => p.Prijs)
14     .ToListAsync();
15
16 // 4. GroupBy: Groepeer resultaten op basis van een sleutel
17 List<IGrouping<decimal, Product>> productenPerPrijs = await
18     _context.Producten
19     .GroupBy(p => p.Prijs)
20     .ToListAsync();
21
22 // 5. First / FirstOrDefault: Haal het eerste element op dat voldoet
23 // aan een voorwaarde
24 Product? eersteDuurProduct = await _context.Producten
25     .FirstOrDefaultAsync(p => p.Prijs > 100);
```

```
26 // 6. Any: Controleer of er ten minste één element voldoet aan een
27 voorwaarde
28 bool heeftDureProducten = await _context.Producten
29     .AnyAsync(p => p.Prijs > 100);
30
31 // 7. All: Controleer of alle elementen voldoen aan een voorwaarde
32 bool alleDuur = await _context.Producten
33     .AllAsync(p => p.Prijs > 10);
34
35 // 8. Count: Tel het aantal elementen dat voldoet aan een voorwaarde
36 int aantalDureProducten = await _context.Producten
37     .CountAsync(p => p.Prijs > 100);
38
39 // 9. Find: Haal een entiteit op op basis van de primaire sleutel
40 Product? product = await _context.Producten.FindAsync(1);
```

SQL

```
1 // 1. Select - specifieke velden (alleen Naam)
2 List<string> namen = await _context.Producten
3     .FromSqlRaw("SELECT Naam FROM Producten")
4     .Select(p => p.Naam)
5     .ToListAsync();
6
7 // 2. Where - filteren op prijs
8 List<Product> dureProducten = await _context.Producten
9     .FromSqlRaw("SELECT * FROM Producten WHERE Prijs > 100")
10    .ToListAsync();
11
12 // 3. OrderByDescending - sorteren
13 List<Product> gesorteerd = await _context.Producten
14     .FromSqlRaw("SELECT * FROM Producten ORDER BY Prijs DESC")
15     .ToListAsync();
16
17 // 4. GroupBy - groeperen (aantal per prijs)
18 var productenPerPrijs = await _context
19     .Set<ProductPrijsGroep>() // Klasse voor Prijs en Aantal
20     .FromSqlRaw(@"
21         SELECT Prijs, COUNT(*) AS Aantal
22         FROM Producten
23         GROUP BY Prijs")
24     .ToListAsync();
```

CS

```
25
26 // DTO-klasse voor groepering
27 public class ProductPrijsGroep
28 {
29     public decimal Prijs { get; set; }
30     public int Aantal { get; set; }
31 }
32
33 // 5. FirstOrDefault - eerste dure product
34 Product? eersteDureProduct = await _context.Producten
35     .FromSqlRaw("SELECT TOP(1) * FROM Producten WHERE Prijs > 100")
36     .AsEnumerable()
37     .FirstOrDefault();
38
39 // 6. Any - check of er dure producten zijn
40 bool heeftDureProducten = await _context.Database
41     .ExecuteSqlRawAsync(@"
42         IF EXISTS (SELECT 1 FROM Producten WHERE Prijs > 100)
43             SELECT 1
44         ELSE
45             SELECT 0") > 0;
46
47 // 7. All - controle of alle producten duurder zijn dan 10
48 bool alleDuur = await _context.Database
49     .ExecuteSqlRawAsync(@"
50         IF NOT EXISTS (SELECT 1 FROM Producten WHERE Prijs <= 10)
51             SELECT 1
52         ELSE
53             SELECT 0") > 0;
54
55 // 8. Count - tel dure producten
56 int aantalDureProducten = await _context.Producten
57     .FromSqlRaw("SELECT * FROM Producten WHERE Prijs > 100")
58     .CountAsync();
59
60 // Of alternatief voor Count met scalar-query
61 // int aantalDureProducten = await _context.Database
62 //     .SqlQueryRaw<int>("SELECT COUNT(*) FROM Producten WHERE Prijs >
63 // 100")
64 //     .SingleAsync();
65
66
67
```

```
68 // 9. Find - ophalen op primaire sleutel
69 Product? product = await _context.Producten
    .FromSqlRaw("SELECT * FROM Producten WHERE Id = {0}", 1)
    .AsEnumerable()
    .FirstOrDefault();
```

API - Toevoegen

In het vorige hoofdstuk leerde je dat een GET-HttpRequest JSON teruggeeft als resultaat. Een API-methode om gegevens toe te voegen, vereist dus ook het ingeven van een JSON object. Dit JSON-object moet overeenkomen met de object-klassen in de parameter van de methode in de controller. De methode in de API-controller heeft een **HttpPost-kenmerk** om aan de API te laten weten dat je gegevens meestuurt. Hieronder zie je een voorbeeld van een methode met kenmerk HttpPost en parameter Product.

LINQ

```
1 [HttpPost]
2 public async Task<ActionResult<Product>> ProductToevoegen(Product
3 product)
4 {
5     //Validatie
6     if (_context.Producten == null)
7         return NotFound("De tabel Producten bestaat niet in de
8 database.");
9     if (product.Naam == "")
10        return BadRequest("Gelieve een naam in te geven");
11    if (product.Prijs == 0)
12        return BadRequest("Gelieve een prijs in te geven");
13
14    //Product toevoegen aan de DbSet
15    await _context.Producten.AddAsync(product);
16    try
17    {
18        //Product wegschrijven naar de database
19        await _context.SaveChangesAsync();
20    }
21    catch (DbUpdateException dbError)
22    {
```

```
23         return BadRequest(dbError);
24     }
    return CreatedAtAction("GetProduct", new {id = product.Id},
product);
}
```

SQL

```
1 [HttpPost]
2 public async Task<ActionResult<Product>> ProductToevoegen(Product
3 product)
4 {
5     // Validatie
6     if (_context.Producten == null)
7         return NotFound("De tabel Producten bestaat niet in de
8 database.");
9     if (string.IsNullOrEmpty(product.Naam))
10         return BadRequest("Gelieve een naam in te geven");
11     if (product.Prijs == 0)
12         return BadRequest("Gelieve een prijs in te geven");
13
14     try
15     {
16         // 📝 Product toevoegen met ruwe SQL
17         // Parameterbinding voorkomt SQL-injectie
18         await _context.Database.ExecuteSqlRawAsync(
19             "INSERT INTO Producten (Naam, Prijs) VALUES ({0}, {1})",
20             product.Naam, product.Prijs);
21
22         // Id ophalen (SQL Server) - SCOPE_IDENTITY() geeft de laatste
23 ID terug
24         var nieuwProductId = await _context.Producten
25             .FromSqlRaw("SELECT TOP(1) * FROM Producten ORDER BY Id
26 DESC")
27             .Select(p => p.Id)
28             .FirstOrDefaultAsync();
29
30         product.Id = nieuwProductId;
31     }
32     catch (DbUpdateException dbError)
33     {
34         return BadRequest(dbError);
35     }
```

```
35     }

    // 201 Created teruggeven met route naar het nieuwe product
    return CreatedAtAction("GetProduct", new { id = product.Id },
        product);
}
```

In het bovenstaande voorbeeld zie je dat er als argument een object van het type `Product` wordt gevraagd (Regel 2). Alle verplichte velden van dit object moet je meegeven in JSON-formaat. Voor een product ziet dat er bijvoorbeeld als volgt uit:

```
1  {
2      "id": 0,
3      "naam": "string",
4      "prijs": 0
5  }
```

json

De id kun je weglaten aangezien dat in de database automatisch wordt genummerd. Als je geen product meegeeft, of je laat één van de verplichte velden leeg, krijg je een *BadRequest* (statuscode 400). Je krijgt dan een foutmelding met welke gegevens ontbreken.

Als je gegevens wil wegschrijven naar een databank is het belangrijk dat je controleert

1. of gegevens kunnen worden weggeschreven naar de databank,
2. of de gegevens correct zijn ingegeven. Gegevensvalidatie wordt vaak over het hoofd gezien maar kan veel fouten opleveren. Je zou het voorbeeld nog kunnen uitbreiden met een controle of het product al bestaat of niet.

Gegevens wegschrijven gebeurt in 2 stappen:

1. Schrijf de gegevens weg naar de DbSet met **AddAsync**.
2. Schrijf de gegevens permanent weg naar de database met **SaveChanges**.

Als laatste stuur je een statuscode 201 (CreatedAtAction) terug naar de gebruiker met het toegevoegde product. De parameters die in de **CreateAtAction**-methode gebruikt worden zijn:

- **Actienaam:** Welke actie kan de nieuwe resource ophalen, in dit geval is dat `GetProduct` (zie vorig hoofdstuk)
- **Route values:** De actie `GetProduct` heeft een id nodig. Deze haal je op uit **product**. Met `SaveChanges` wordt het product toegevoegd aan de database en wordt de `Id`

aangemaakt. Deze Id wordt dan teruggegeven aan **product**.

- **Response content:** Het object dat als body van de response wordt teruggestuurd naar de gebruiker.

API - Wijzigen (update)

Het wijzigen van een product kan met onderstaand voorbeeld. Je gebruikt nu een HttpPut-kenmerk om aan te geven dat het om updaten gaat.

LINQ

```
1  [HttpPut("{id}")]
2  public async Task<ActionResult> ProductWijzigen(int id, Product
3  product)
4  {
5      if (id != product.Id)
6      {
7          return BadRequest("De opgegeven id's komen niet overeen.");
8      }
9      //Hier kan nog andere validatie komen.
10
11     _context.Producten.Update(product);
12
13     try
14     {
15         await _context.SaveChangesAsync();
16     }
17     catch (DbUpdateConcurrencyException)
18     {
19         if (await _context.Producten.AnyAsync(x => x.Id == id))
20         {
21             return NotFound("Er is geen product met dit id gevonden");
22         }
23         else
24         {
25             throw;
26         }
27     }
28
29 
```

```
    }  
}  
  
return NoContent();  
}
```

SQL

```
1  [HttpPut("{id}")]                                     CS  
2  public async Task<ActionResult> ProductWijzigen(int id, Product  
3  product)  
4  {  
5      // Basisvalidatie  
6      if (id != product.Id)  
7      {  
8          return BadRequest("De opgegeven id's komen niet overeen.");  
9      }  
10  
11     // Optioneel: extra validatie  
12     if (string.IsNullOrEmpty(product.Naam))  
13     {  
14         return BadRequest("Gelieve een geldige naam in te geven.");  
15     }  
16     if (product.Prijs <= 0)  
17     {  
18         return BadRequest("Gelieve een geldige prijs in te geven.");  
19     }  
20  
21     try  
22     {  
23         // Controleer eerst of het product bestaat  
24         var bestaat = await _context.Producten  
25             .FromSqlRaw("SELECT * FROM Producten WHERE Id = {0}", id)  
26             .AnyAsync();  
27  
28         if (!bestaat)  
29         {  
30             return NotFound("Er is geen product met dit id gevonden.");  
31         }  
32  
33         // Update uitvoeren met ruwe SQL  
34         await _context.Database.ExecuteSqlRawAsync(  

```

```
35         "UPDATE Producten SET Naam = {0}, Prijs = {1} WHERE Id =  
36         {2}",  
37         product.Naam, product.Prijs, id);  
38     }  
39     catch (DbUpdateException dbError)  
40     {  
41         return BadRequest(dbError);  
42     }  
43  
    return NoContent();  
}
```

Als argument geef je in de URL de id mee van het product. In de body moet je dezelfde id opgeven. Als deze niet overeenkomen, krijg je een **BadRequest** met een gepaste foutmelding. Daarna volgt eventueel validatie van je gegevens.

De volgende stap is het wijzigen van de gegevens in de DbSet, gevolgd door het proberen wijzigen van de gegevens in de database. Merk op dat er voor Update **geen** async-methode bestaat. Het is interessant om dit te doen met een **try-catch** aangezien je op die manier kunt controleren of het product ondertussen al niet door een andere gebruiker gewijzigd wordt. **DbUpdateConcurrencyException** gaat deze uitzondering werpen.

In het catch-gedeelte controleer je eerst of het product nog bestaat. Indien het niet meer bestaat, geef je een 404 Not found terug. In het andere geval is er een ander onverwacht probleem opgetreden.

Als alles succesvol is verlopen, geeft de methode een 204 No Content-status terug, wat aangeeft dat de update succesvol was, maar er geen inhoud is om terug te sturen.

API - Verwijderen

Het verwijderen van een product kan met onderstaand voorbeeld. Je gebruikt nu een **HttpDelete**-kenmerk om aan te geven dat het om verwijderen gaat. Je geeft de id van het product mee in de URL.

LINQ

```
1 [HttpDelete("{id}")]
2 public async Task<IActionResult> ProductVerwijderen(int id)
3 {
4     if (_context.Producten == null)
5     {
6         return NotFound("De tabel producten bestaat niet.");
7     }
8     Product product = await
9     _context.Producten.FirstOrDefaultAsync(x=>x.Id == id);
10    if (product == null)
11    {
12        return NotFound("Het product met deze id is niet gevonden.");
13    }
14
15    _context.Producten.Remove(product);
16    await _context.SaveChangesAsync();
17
18    return Ok($"Product met id {id} is verwijderd");
19 }
```

SQL

```
1 [HttpDelete("{id}")]
2 public async Task<IActionResult> ProductVerwijderen(int id)
3 {
4     // Check of de DbSet bestaat
5     if (_context.Producten == null)
6     {
7         return NotFound("De tabel producten bestaat niet.");
8     }
9
10    // Controleer of het product bestaat
11    bool bestaat = await _context.Producten
12        .FromSqlRaw("SELECT * FROM Producten WHERE Id = {0}", id)
13        .AnyAsync();
14
15    if (!bestaat)
16    {
17        return NotFound("Het product met deze id is niet gevonden.");
18    }
19
20    try
```

```
20     {
21         // Verwijder het product met ruwe SQL
22         await _context.Database.ExecuteSqlRawAsync(
23             "DELETE FROM Producten WHERE Id = {0}", id);
24     }
25     catch (DbUpdateException dbError)
26     {
27         return BadRequest(dbError);
28     }
29
30     return Ok($"Product met id {id} is verwijderd");
31 }
32
```

In dit voorbeeld wordt eerst gecontroleerd of de tabel Producten bestaat en of de id kan worden gevonden. Daarna wordt het product verwijderd in de DbSet en vervolgens permanent verwijderd uit de database. Merk op dat er voor Remove **geen** async-methode bestaat. Je geeft een gepaste melding terug als het product succesvol verwijderd is. Je kunt hier ook weer een try-catch gebruiken om te controleren of er gelijktijdig geen wijzigingen gebeuren aan het product.

Validatie en data-annotaties

Het is ondertussen duidelijk dat gegevensvalidatie heel belangrijk is bij het ingeven en wijzigen van gegevens. In grotere models kan dat echter voor een heel lange reeks van commando's resulteren. Controleer of de naam ingevuld is, of de naam niet langer is dan 50 tekens...

Een eerste oplossing is om een deel van die checks in een aparte functie te zetten om dubbele code te vermijden. Een betere oplossing is om te werken met data-annotaties. Dat zijn kenmerken die je bij je properties in het model plaatst, met gepaste foutmeldingen. In je methoden hoef je dan enkel nog te controleren of je inkomende gegevens voldoen aan de regels van je model. Hieronder is een voorbeeld voor het model Product.

```
1 public class Product
2 {
3     public int Id { get; set; }
4
5     [Required(ErrorMessage = "De naam moet ingevuld worden.")]
6     [MinLength(3, ErrorMessage = "De naam moet minstens 3 tekens
```

CS

```
7     bevatten."]]
8     [StringLength(100, ErrorMessage = "De naam mag niet meer dan 100
9     tekens bevatten.")]
10    public string Naam { get; set; }
11
12    [StringLength(500, ErrorMessage = "De beschrijving mag niet langer
13    zijn dan 500 tekens.")]
14    public string? Beschrijving { get; set; }
15
16    [Required(ErrorMessage = "De prijs van het product is verplicht.")]
17    [Range(0.01, 10000.00, ErrorMessage = "De prijs moet tussen 0,01 en
18    10.000 liggen.")]
19    public decimal Prijs { get; set; }
20 }
```

Standaard zijn properties verplichte velden en is het kenmerk *[Required]* overbodig. Als je echter aangepaste foutmeldingen wil weergeven, moet je het wel boven je Property toevoegen.

In plaats van `StringLength` kun je ook `MaxLength` gebruiken. Gebruik **`StringLength`** wanneer je wilt **valideren** hoe lang een string mag zijn voordat deze naar de database wordt geschreven. Gebruik **`MaxLength`** wanneer je wilt **beperken** hoe lang een string daadwerkelijk in de databasekolom mag zijn.

Denk eraan dat elke wijziging aan je model ook vereist dat je een migratie aanmaakt en je database updatet. Let op dat wijzigingen aanbrengen in je database kan resulteren dat er gegevens verloren gaan. Een goede data-analyse voor je begint te programmeren is altijd wenselijk.

In het voorbeeld is, naast de data-annotaties, ook een extra property `Beschrijving` toegevoegd. Om de wijzigingen door te voeren gebruik je volgende commando's:

```
1  \\ Migratie aanmaken                                pwsh
2  add-migration "Added Beschrijving to Product"
3  \\ Database updaten
4  update-database
```

In je methode kun je de validatie van je model testen op volgende eenvoudige manier:

```
1     if (!ModelState.IsValid)
2     {
3         return BadRequest(ModelState);
4     }
```

CS

Als antwoord krijg je dan de foutmeldingen voor al de voorwaarden waaraan niet voldaan is!

The screenshot shows a web browser window with the URL `https://localhost:7018/api/Product`. Below the address bar, it says "Server response". A table with two columns, "Code" and "Details", shows a 400 status code. Below the table, the "Response body" is displayed as a JSON object. The JSON object contains a "type" pointing to an IETF RFC section, a "title" stating "One or more validation errors occurred.", a "status" of 400, and an "errors" object with two entries: "Naam" (Name) and "Prijs" (Price), each with a specific validation error message. A "traceId" is also included at the bottom of the JSON object.

Code	Details
400 <i>Undocumented</i>	Error: response status is 400

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Naam": [
      "De naam moet minstens 3 tekens bevatten."
    ],
    "Prijs": [
      "De prijs moet tussen 0,01 en 10.000 liggen."
    ]
  },
  "traceId": "00-a154ce2cbbd328295fa756c3782725f9-1e28c610c87829ea-00"
}
```

! ModelState is een concept uit ASP.NET Core MVC en Razor Pages en wordt gebruikt om server-side modelvalidatie en foutmeldingen bij te houden. In .NET MAUI en Blazor werkt het iets anders:

- Server-side Blazor draait bovenop ASP.NET Core, maar gebruikt geen ModelState-object. In plaats daarvan gebruik je EditForm met data-annotaties ([Required], [StringLength], enz.) en een ValidationSummary of ValidationMessage. Validatie gebeurt via de ingebouwde DataAnnotationsValidator. [Meer info over Data-annotaties in Blazor.](#)
- In Maui kun je de communitytoolkit.mvvm gebruiken in combinatie met data-annotaties. In de communitytoolkit zit een methode **observablevalidator** die je UI kan updaten bij errors tijdens het valideren. [Meer info over Data-annotaties in MAUI.](#)

[Hier vind je het WebApiDemo-project.](#)

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM